PHASE 7

Project Documentation

Consolidated overview, installation guide, and walkthrough of the finished application

PulseView – Heart Disease Risk Analysis Dashboard

Project	PulseView – Heart Disease Risk Analysis Dashboard
Document	Final Project Documentation
Version	1.0
Date	July 2026
Prepared By	Project Team

1. Project Overview

PulseView is a Flask web application that presents a heart-disease risk-factor dataset through an embedded Tableau dashboard and a guided four-scene story. It was built for three audiences — clinicians, health-policy staff, and individuals tracking their own risk — so that demographic, lifestyle, and clinical patterns in the data can be explored without any Tableau installation or statistics background.

This document consolidates the outcome of all six preceding project phases (Ideation, Requirement Analysis, Design, Planning, Development, and Performance Testing) into a single reference for installing, running, and understanding the finished application.

2. Feature Summary

Page	Route	Purpose
Home	/	Landing page introducing the project and linking to the story and dashboard.
About	/about	Problem statement, the three user personas, and the six-step build process.
Dashboard	/dashboard	Embedded Tableau dashboard: Gender vs. Heart Disease, Diabetic vs. Stroke, Race-wise Heart Disease, Smoking & Alcohol Impact.
Story	/story	Embedded four-scene guided story: Overview, Gender Risk, Lifestyle Factors, Race & Conditions.
Contact	/contact	Validated contact form with flash-message feedback.

3. System Requirements

- Python 3.x
- pip (Python package manager)
- A modern web browser (Chrome, Firefox, Edge, or Safari)
- Internet access, so the browser can load the Tableau Public embeds

4. Installation & Setup

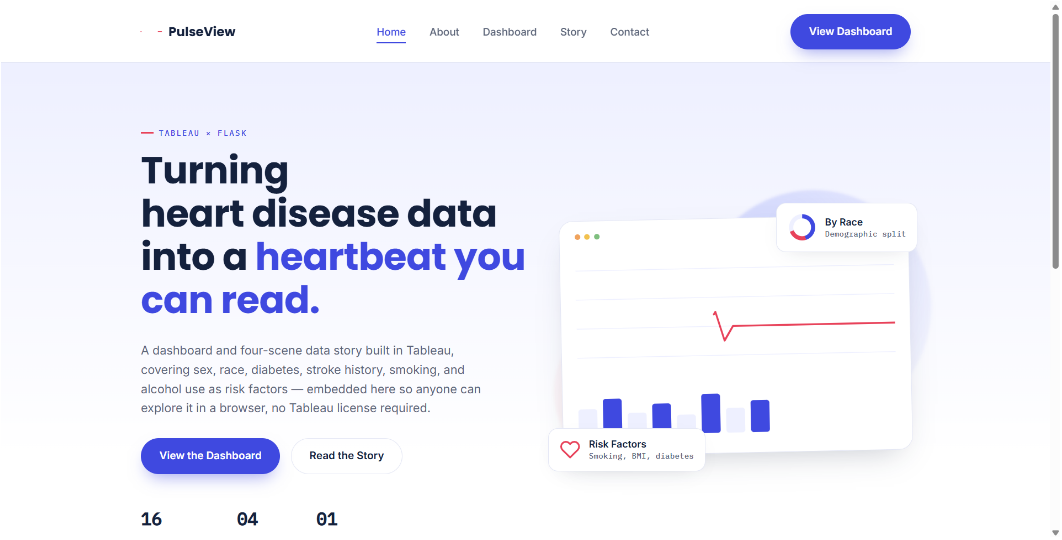
From the project's Flask/ directory, run:

```
python -m venv venv
source venv/bin/activate        # Windows: venv\Scripts\activate
pip install -r requirements.txt
python app.py
```

Then open `http://127.0.0.1:5000` in a browser. requirements.txt pins a single dependency, `Flask==3.1.3`.

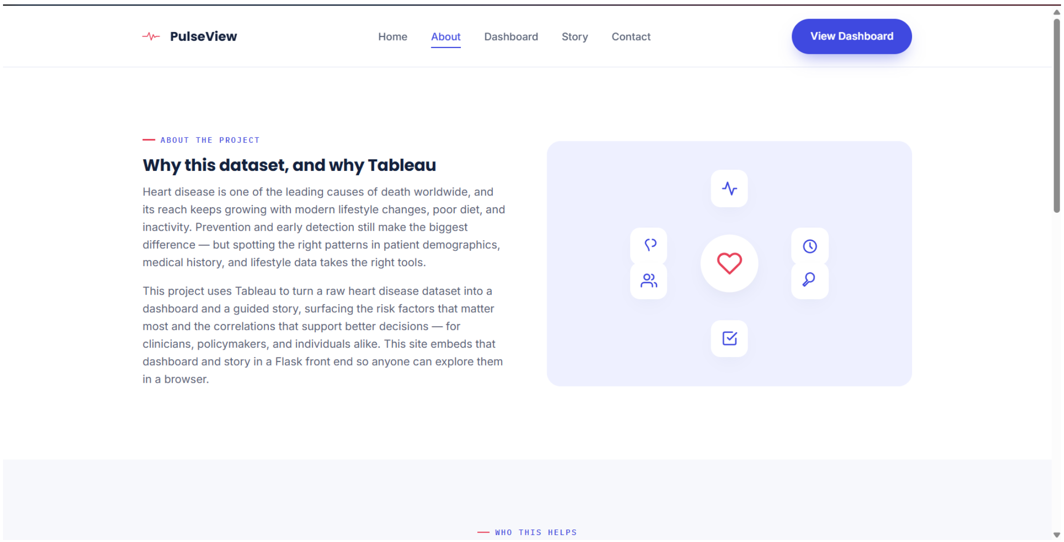
5. Application Walkthrough

5.1 Home



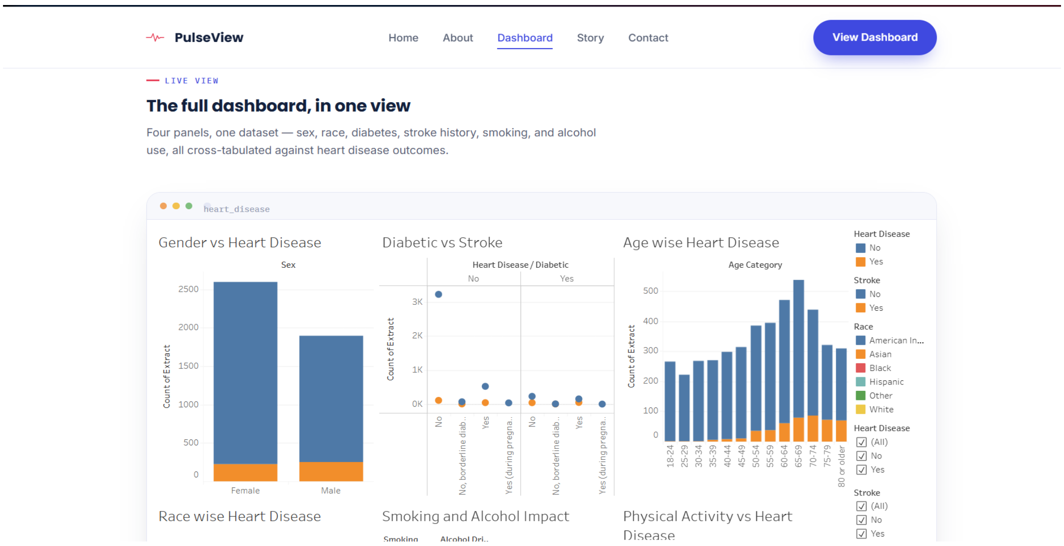
The landing page, with entry points to the Story and Dashboard.

5.2 About



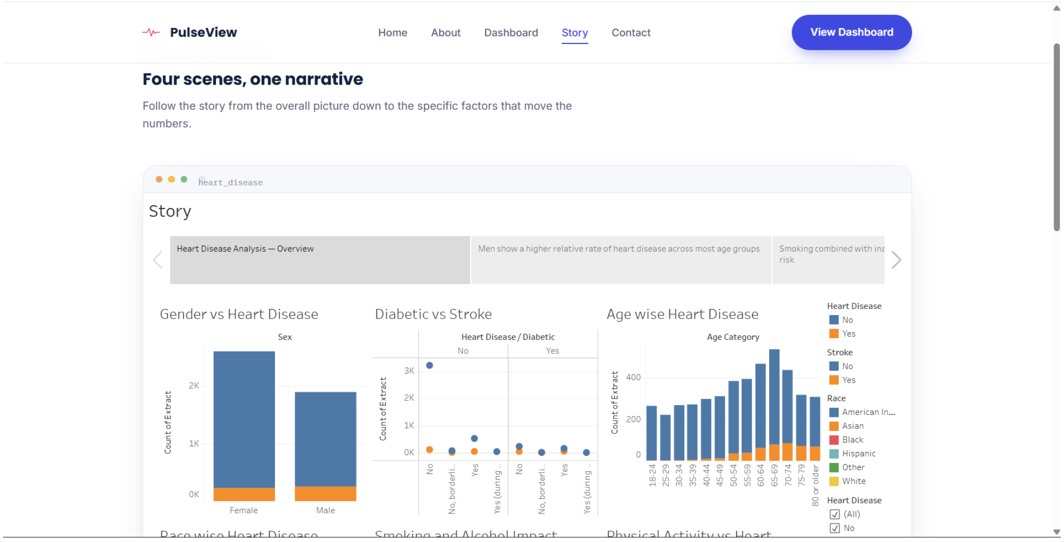
Problem framing, personas, and build process.

5.3 Dashboard



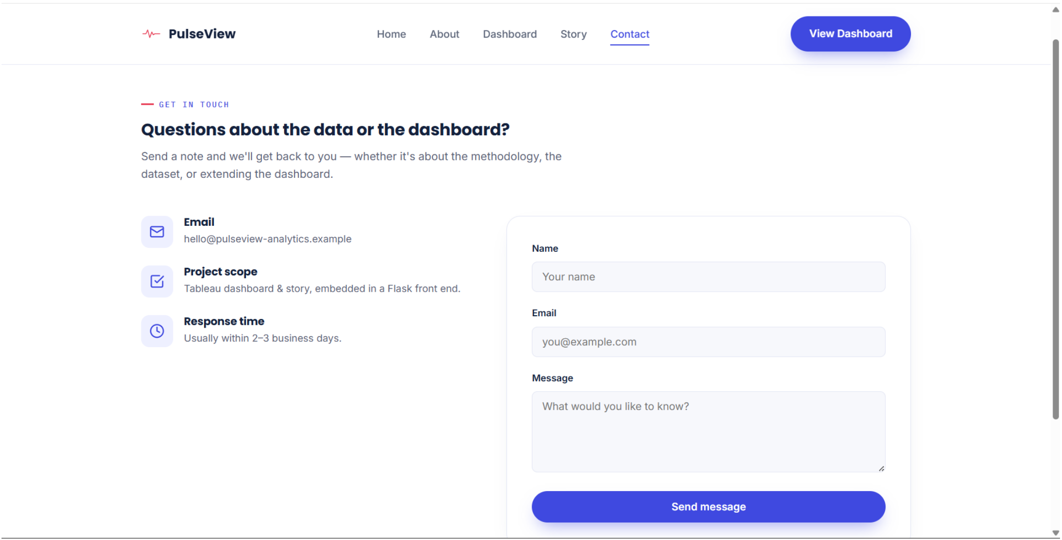
The embedded Tableau dashboard with its four panels.

5.4 Story



The four-scene guided story with tab and prev/next navigation.

5.5 Contact



The validated contact form.

6. Project Structure

```
heart_disease_flask/
■■■ app.py                 # Routes + STORY_SCENES / DASHBOARD_PANELS data
■■■ requirements.txt
■■■ static/
■   ■■■ css/style.css
■   ■■■ js/main.js          # Story tab/prev-next/keyboard navigation
■   ■■■ images/             # dashboard.png + 4 story scene PNGs
■■■ templates/
    ■■■ base.html          # Shared layout, nav, footer
    ■■■ index.html
    ■■■ about.html
    ■■■ dashboard.html
    ■■■ story.html
    ■■■ contact.html
```

To point the site at a new Tableau workbook, update `TABLEAU_DASHBOARD_EMBED_URL` and `TABLEAU_STORY_EMBED_URL` in app.py; to add or edit scenes and panels, edit the `STORY_SCENES` / `DASHBOARD_PANELS` lists and drop new images into `static/images/`. No template changes are required for either.

7. Known Limitations & Future Scope

Limitation	Future Scope
Contact form has no email backend wired up.	Integrate Flask-Mail or a transactional email API to actually deliver submitted messages.
No user accounts or saved views.	Add lightweight session-based bookmarking of dashboard filters for returning visitors.
Dashboard/story availability depends on Tableau Public uptime.	Evaluate a self-hosted Tableau Server or an alternative embeddable BI tool for production use.
All data is static/aggregate; no live data refresh.	Connect Tableau to a live data source with a scheduled extract refresh.

8. Conclusion

PulseView successfully packages a Tableau-driven analysis of heart-disease risk factors into an accessible, browser-based experience for clinicians, policymakers, and individual users alike. The project moved through six planned phases — ideation, requirement analysis, design, planning, development, and performance testing — each documented separately, with this document serving as the consolidated reference for installing, running, and extending the finished application.